Qualité logicielle

La qualité ici, c'est du code qui reste constant, simple et peu complexe — imposé par l'outillage en CI, pas par la mémoire. Les tests et la couverture sont traités à part dans Tests ; ce handbook parle de **style, conventions et analyse statique**.

Back-end — Checkstyle

L'API est vérifiée contre une config **Checkstyle** sur mesure (`checkstyle.xml`) couvrant le nommage (`TypeName`, `MethodName`, `MemberName`, `ConstantName`), les imports (pas d'import en étoile, pas d'import inutile ou redondant), les espaces, les accolades (`NeedBraces`), l'ordre des modificateurs et l'indentation. Les lignes sont plafonnées à **120** caractères.

Elle plafonne aussi la complexité, pour que les méthodes restent courtes et lisibles :

Règle	Limite
Longueur de méthode	60 lignes
Paramètres par méthode	7
Complexité cyclomatique	10
Complexité NPath	200
Longueur d'une classe interne anonyme	20 lignes

Un `checkstyle-suppressions.xml` associé relâche quelques règles là où elles ajoutent du bruit : les contrôleurs sont dispensés du Javadoc sur méthodes (les annotations Spring parlent d'elles-mêmes), les entités peuvent utiliser des nombres magiques, et les mappers MapStruct générés, les contrôleurs et les sources de test sont exemptés des contrôles de longueur de ligne et d'indentation.

Checkstyle tourne en CI (`mvn … checkstyle:checkstyle`) et le rapport HTML est publié en artefact de build.

Front-ends — ESLint & Prettier

Les deux applications Angular passent le lint avec **ESLint 9** via `@angular-eslint` et `@typescript-eslint`, et le formatage avec **Prettier** :

```
npm run lint      # ng lint (ESLint)
npm run format    # prettier --write .
```

`npm run lint` tourne en CI pour l'app comme pour le dashboard. Un `.editorconfig` fixe les bases communes — UTF-8, indentation à **2 espaces**, une ligne finale, les espaces de fin supprimés, et les **guillemets simples** en TypeScript. Le dashboard ajoute quelques plugins ESLint supplémentaires (`import`, `jsdoc`, `prefer-arrow`). L'app mobile dispose de **husky** pour les hooks Git, même si le lint tourne pour l'instant manuellement et en CI plutôt qu'au commit.

Analyse statique

Deux couches d'analyse statique s'ajoutent au lint.

DeepSource tourne en continu sur les trois dépôts (`.deepsource.toml`) :

- **API** — l'analyzer Java (runtime 21), avec `test_patterns` restreint à `**/*Test.java` pour que les fixtures partagées (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO`, `TestHelper`) ne soient pas signalées comme fichiers de test vides.
- **App mobile** — l'analyzer JavaScript avec le plugin Angular (environnements Node, Jasmine, Cypress) ; deux règles sont ignorées là où les patterns Angular les déclenchent à tort (classes de module vides `JS-0327`, aliasing d'input requis `JS-0585`).
- **Dashboard** — l'analyzer JavaScript avec le plugin Angular (Node, Jasmine).

JetBrains Qodana ajoute une seconde passe (le profil `qodana.starter`), mais seulement sur l'**API** (`qodana_code_quality.yml`, linter JVM community). Côté app mobile un workflow Qodana existe mais est **désactivé** (`.yml.disabled`), et le dashboard n'en a pas. Les seuils du quality gate de Qodana dans `qodana.yaml` sont laissés en commentaire, donc il remonte ses findings sans faire échouer le build sur un décompte *(à confirmer)*.

Où la qualité est imposée

Dépôt	Style	Analyse
`candex-api`	Checkstyle	DeepSource + Qodana
`candex-app`	ESLint + Prettier	DeepSource
`candex-dashboard`	ESLint + Prettier	DeepSource

L'API a aussi la barrière JaCoCo 100 % — voir Tests. Le lint et Checkstyle tournent en CI à chaque push et pull request, DeepSource analyse chaque changement en continu, et un changement n'est pas terminé tant que tout n'est pas vert. Pour la vue complète de la CI et les barrières tests/couverture, voir Tests.

Pour aller plus loin

- Tests et couverture → Tests
- Le code auquel ces règles s'appliquent → Back-end • Front-end